

Quantor-MTFuzz Roadmap: Completed vs Pending

Legend

Completed

Pending

Planned (Next)

dashed = optional/parallel

Completed Work

Backtesting Architecture
(PositionState + synthetic options)

Mark-to-Market P&L
+ Leg exit logic

Trade Logging + Diagnostics

Professional Reporting
(PDF, visuals, zero-trade fix)

Phased Serial Optimizer
+ Metrics + CSV/top100

CPU Benchmark

AlpacaBroker Integration
+ live/paper wiring

Docs & Roadmap

Stage 0-1: Baseline & Data Realism

Baseline Metrics
(PnL/D/Sharpe + runtime)

Contig Sanity Checks
(main.py)

Add risk_free_rate
+ iv_annual_volatility

Realized Vol + IV Rank Proxy

Slippage + Commission Model

Stage 2-3: Strategy Depth & Risk Controls

SkewTerm Structure
Strike Selection

Probabilistic Entry Filter

Regime Classifier
(Irend vs Mean Reversion)

Portfolio Greeks
(Delta/Gamma/Vega)

Portfolio Risk Caps
+ Risk Budget Sizing

Shock/Stress tests

Stage 4-5: Performance & Validation

Parallel Optimizer

Pre-index Option Chains

Hot-loop Optimizations
(lookup + vectorize/JIT)

Expanded PDF Metrics
(DTE/Credit/Regime)

Regression + Unit Tests

Executive Summary

Quantor-MTFuzz is a specialized SPY Iron Condor system designed to unify high-fidelity backtesting, parameter optimization, and live/paper execution under a single, disciplined workflow. The core design emphasizes realistic mark-to-market accounting, multi-timeframe signal filtering, and dynamic position sizing to control risk while maintaining operational readiness for live deployment. The system already incorporates a robust trade lifecycle, professional reporting, and a functioning optimization engine, with a completed backlog of critical stability, performance, and reporting milestones.

The current architecture cleanly separates responsibilities: `main.py` orchestrates runs, `backtest_engine.py` performs high-resolution simulation, `optimizer.py` handles phased grid search, and `options_strategy.py` encapsulates strategy logic and trade management. Intelligence modules (fuzzy sizing and regime filters) provide higher-order context, while data modules support both synthetic and live workflows. This structure is consistent with production-ready trading research platforms and makes future enhancements tractable.

Key strengths today include a leg-accurate backtest engine with MTM P&L, coherent logging, and visual reporting; a serial optimizer with caching and CSV top-100 outputs; live/paper execution via Alpaca integration; and documented math foundations and operational workflow. The system can already be used end-to-end for research, parameter tuning, and paper deployment.

The primary gaps now are advanced market realism (slippage/commission modeling, IV/VIX realism, American exercise effects), deeper strategy sophistication (skew/term-structure strike selection, probabilistic risk filters, regime classification), and compute optimization (parallelized grid search and faster option-chain lookups). These are well-defined, high-impact enhancements that will improve both backtest fidelity and live robustness.

A staged plan has been articulated to deliver improvements in a controlled sequence: establish baselines and safety checks; raise data realism; deepen strategy logic; add portfolio-level risk controls; optimize compute performance; and finally expand reporting and validation. This plan preserves system stability while enabling meaningful performance gains and more defensible research outcomes.

Overall, Quantor-MTFuzz is in a strong “research-ready” and “paper-ready” state. With the proposed staged upgrades, it can evolve into a higher-fidelity, institution-style options research and execution framework, with improved realism, stronger risk governance, and faster iterative research cycles.

The github repository and its `readme.md` and other `.md` files have been updated to current.

Category	Task Description	Status
Backtesting Architecture	Implement leg-based data structure for strategy (<code>PositionState</code>)	 Completed
Backtesting Architecture	Integrate synthetic options data into <code>backtest_engine</code>	 Completed
P&L Accounting	Implement mark-to-market P&L calculation	 Completed
Trade Management	Implement individual leg exit logic (profit-target, stop-loss, expiration)	 Completed
Trade Management	Enhance trade logging with full multi-leg details	 Completed
Diagnostics	Add diagnostic entry/exit logging	 Completed
System Validation	Verify end-to-end trading lifecycle (2 trades, \$160 net profit)	 Completed
Reporting	Refine reporting logic (fix date display for zero-trade runs)	 Completed
Data Robustness	Improve data resilience (handle leg price gaps / “N/A”)	 Completed
Visualization	Finalize professional reporting visualizations	 Completed
Optimization	Run grid-search optimization (maximize profit / drawdown ratio)	 Completed
Performance	Execute CPU benchmark (<code>core/benchmark_cpu.py</code>)	 Completed
Optimization Engine	Implement phased serial optimizer in <code>core/optimizer.py</code>	 Completed
Stability	Fix Backtrader crash and implement data caching	 Completed
Metrics	Add Sharpe ratio, balance metrics, and rank-based parameter application	 Completed
Broker Integration	Implement <code>AlpacaBroker</code> for paper and live trading	 Completed
Live Trading	Finalize <code>alpaca-trade-api</code> integration in <code>core/main.py</code>	 Completed
Live Trading	Conduct Alpaca test account verification run	 Completed
Delivery	Prepare final walkthrough and supporting artifacts	 Completed

Category	Task Description	Status
Version Control	Synchronize repository with GitHub	 Completed
Collaboration	Add <code>CONTRIBUTING.md</code> for team collaboration	 Completed
Documentation	Document resolved bugs and development roadmap	 Completed
Optimization Metrics	Expand optimizer metrics (profit factor, wins, losses, E-ratio)	 Completed
Optimization Reporting	Implement timestamped optimizer reporting (CSV)	 Completed
Documentation	Update <code>README.md</code> with optimizer report details	 Completed
Documentation	Document performance metrics mathematics in <code>README.md</code>	 Completed
Debugging	Debug optimizer application crash (<code>os NameError</code>)	 Completed
Version Control	Synchronize final repository updates	 Completed
Architecture Docs	Document dual-data architecture and 4-leg execution in <code>README.md</code>	 Completed
Reporting	Round optimizer metrics in CSV and terminal output	 Completed
Deployment	Apply Rank-1 configuration (<code>profit_take_pct = 0.9</code>)	 Completed

This where we are now with Quantor-MTFuzz™.

The following shows all files, how they interact, what the purpose of the system is, and how we can improve its performance by adding additional sophisticated trading techniques, mathematics, physics, etc.

Purpose and flow

This project is an SPY Iron Condor system for backtesting, optimization, and live/paper execution with multi-timeframe (MTF) context and fuzzy sizing. The flow is: main.py builds StrategyConfig/RunConfig → backtest mode calls backtest_engine.py (which uses sync_engine.py, options_strategy.py, and fuzzy_engine.py) → optimizer mode calls optimizer.py (which loops run_backtest_headless) → live/paper mode uses broker.py with polygon_client.py and executes logic in options_strategy.py.

File report (all files)

- Root
 - README.md project overview, usage, CLI flags, and file glossary.
 - CONTRIBUTING.md contribution workflow and setup instructions.
 - DEVELOPMENT_STATUS.md resolved bugs and roadmap.
 - requirements.txt Python dependencies.
 - PolyOptionsData.py Polygon options data scraper and timeframe aggregation.
 - .gitignore git ignores.
- core/
 - main.py CLI entrypoint; builds config, dispatches backtest/optimizer/live.
 - backtest_engine.py high-fidelity backtest with MtM PnL, reporting, and PDF.
 - optimizer.py serial grid search with caching and top-100 reporting.
 - broker.py broker abstractions + PaperBroker/AlpacaBroker implementations.
 - liquidity_gate.py live-only spread/credit gate (currently unused in strategy).
 - benchmark_cpu.py hardware benchmark and runtime projection.
 - trade_decision.py dataclass for trade decisions (used by paper executor only).
 - config.template.py default StrategyConfig/RunConfig template.
 - __init__.py package marker.
- strategies/
 - options_strategy.py condor construction, entry filters, exit logic, and hedging.
 - __init__.py package marker.

- intelligence/
 - fuzzy_engine.py fuzzy sizing, MTF/IV/VIX memberships.
 - regime_filter.py liquidity/volatility gate (used by strategy).
 - __init__.py package marker.
 - __init__.py package marker.
 - __init__.py package marker.
- analytics/
 - metrics.py simple trade summary + plot for live/paper.
 - audit_logger.py slippage log helper.
 - __init__.py package marker.
- data_factory/
 - polygon_client.py stubbed Polygon client and chain converter.
 - AlpacaGetData.py Alpaca bar downloader and interpolation.
 - SyntheticOptionsEngine.py Black-Scholes synthetic chain generator.
 - sync_engine.py MTF data alignment for signal snapshots.
 - __init__.py package marker.
- execution/
 - paper_executor.py simple trade lifecycle store (not wired to main flow).
- analytics/, data/, reports/, strategies/, intelligence/ directories provide outputs and module structure; data/ and reports/ are data/outputs and may be empty in repo.

How files interact (key touch points)

- Backtest path: main.py → backtest_engine.py → options_strategy.py (build/price condor) + fuzzy_engine.py (sizing) + sync_engine.py (MTF snapshots) → outputs reports/.
- Optimizer path: main.py → optimizer.py → backtest_engine.py with cached data + outputs to top100_*.csv.

- Live/paper path: main.py → broker.py (Paper/Alpaca)
+ polygon_client.py → options_strategy.py for entries/exits/hedging → metrics.py for visualization.

Notable observations

- SyntheticOptionsEngine.py expects RunConfig.risk_free_rate and RunConfig.iv_annual_volatility, which are not present in config.template.py.
- liquidity_gate.py is not used by options_strategy.py; strategy uses regime_filter.py instead.
- Live data sources are stubbed in polygon_client.py; backtest uses synthetic options in data/synthetic_options.
- Docs and PolyOptionsData.py include mojibake characters, likely encoding issues.

Performance + sophistication improvements (prioritized)

Data/market realism

- Replace stub IV/VIX in backtest with realized measures from price/vol data and a proper IV surface per date.
- Add slippage/commission and bid-ask modeling in backtest_engine.py to reduce optimistic bias.
- Use American option pricing (binomial/finite-difference) or LSM for early-exercise effects vs pure Black-Scholes.

Advanced trading techniques

- Add skew/term-structure-aware wing selection: choose strikes using delta-skew or probability-weighted expected shortfall.
- Portfolio-level Greeks and risk caps: delta/gamma/vega limits across concurrent positions.
- Dynamic hedging rules beyond delta (vanna/volga) with regime-dependent triggers.

Mathematics/physics-inspired modeling

- Stochastic volatility (Heston/SABR) or local volatility calibration for synthetic data.
- Jump-diffusion or regime-switching models (e.g., OU regime for volatility) to capture tail risk.

- Kalman filter or particle filter for latent regime detection (trend vs mean-reversion) to gate entries.

Optimization and compute performance

- Parallelize grid search in optimizer.py (multiprocessing or joblib).
- Pre-index option chains by date/strike/expiry to avoid per-bar list scans.
- Vectorize or JIT (Numba) the hot loops in backtest_engine.py.

Task list for every stage

Stage 0 — Baseline & Safety

- Add a small benchmark runner that prints runtime per 10k bars and stores it in benchmark.txt (benchmark_cpu.py).
- Capture baseline metrics (PnL/DD/Sharpe/runtime) and save to baseline_metrics.json after a backtest run (backtest_engine.py).
- Document baseline run command in README.md under a “Benchmark” section.
- Add a simple config sanity check (missing keys, invalid ranges) before running (main.py).

Stage 1 — Data & Market Realism

- Add risk_free_rate and iv_annual_volatility to RunConfig (config.template.py).
- Update SyntheticOptionsEngine.py to handle missing fields gracefully and default to config values.
- Replace random IVR/VIX in backtest with computed series:
 - Add a realized volatility calculator from 5m bars (backtest_engine.py).
 - Add a simple IV rank proxy using rolling realized vol percentile (backtest_engine.py or metrics.py).
- Add bid/ask slippage model in backtest:
 - Apply spread-based slippage on entry/exit costs (backtest_engine.py).
- Add commission model (per contract, per leg) in backtest (backtest_engine.py).
- Document assumptions in README.md.

Stage 2 — Strategy Depth

- Add skew/term-structure aware strike selection:

- Extend OptionQuote to include optional gamma/vega if present in synthetic data (options_strategy.py).
- Modify nearest_by_delta to include IV skew penalty (options_strategy.py).
- Add probabilistic entry filter:
 - Compute breach probability using delta and time to expiration (simple normal approximation) (options_strategy.py).
 - Reject entries above a threshold (config param in StrategyConfig).
- Add a regime classifier:
 - Implement trend vs mean-reversion score from MTF data (fuzzy_engine.py or new regime_classifier.py).
 - Gate entries based on regime (new config threshold).
- Add conditional wing-width logic based on skew/regime (options_strategy.py).

Stage 3 — Risk & Portfolio Controls

- Track portfolio Greeks across open positions:
 - Add a compute_greeks helper for positions using option chain (options_strategy.py).
 - Store aggregate delta/gamma/vega in broker metrics (broker.py).
- Add portfolio-level risk caps:
 - New config limits (max abs delta/gamma/vega) in StrategyConfig.
 - Enforce before entry and before roll (options_strategy.py).
- Add risk-budget sizing:
 - Extend fuzzy sizing to include portfolio-level risk budget (fuzzy_engine.py).
- Add stress tests:
 - Create a simple shock scenario runner ($\pm 2-4\%$ underlying, $+30\%$ IV) (backtest_engine.py or metrics.py).

Stage 4 — Performance & Compute

- Parallelize optimizer:

- Replace serial loop with multiprocessing/joblib; keep deterministic seed (optimizer.py).
- Cache option chains more efficiently:
 - Build a nested dict {date: {expiry: {strike: record}}} (backtest_engine.py).
- Optimize hot loops:
 - Replace list scans with pre-indexed lookups in build_condor and pricing (options_strategy.py + backtest_engine.py).
- Optional JIT/Numba for PnL loop if still slow (only after profiling).

Stage 5 — Reporting & Validation

- Expand PDF report:
 - Add distribution plots for DTE, credit/width, and regime label (backtest_engine.py).
- Add tests:
 - Create test_pnl.py for PnL and drawdown sanity.
 - Create test_strategy_filters.py for IVR/VIX/regime gating.
- Add a regression run:
 - Save a small deterministic backtest dataset and expected metrics (fixed seed) (data/ + tests/).
- Update README.md for github repo with new metrics and validation instructions.